

Grad Café SQL Analysis: Query Results

JHU EN.605.256 Modern Software Concepts in Python · Module 4 · results from September 27, 2026

Database: 30,503 entries, added between January 15, 2026 and September 15, 2026.

How averages are calculated: a score only counts if it is on its official scale. That means a GPA above 0 and no higher than 4.0, GRE Verbal and Quantitative between 130 and 170, and Analytical Writing between 0 and 6. This matters because 1,459 values in the GRE Quantitative column are off that scale, and 1,427 of them are 260–340 total scores that applicants typed into the section-score field. Averaging the column as it is stored would give a "GRE Quantitative" of 259.96. The same rule also leaves out 84 Analytical Writing values and 135 GPAs. None of the stored values are changed; off-scale values are only left out of the averages and of the counts shown next to them.

How text is matched: comparisons ignore capital letters and extra spaces at the start or end. An acceptance is any status that starts with "accept". The regular expressions use PostgreSQL's ~* operator, which ignores case, and \m and \M mark the start and end of a whole word.

Question 1

How many entries in the database are from applicants who applied for Fall 2026?

RESULT

Fall 2026 applicant count: **30,067**

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q1 = r"""
SELECT COUNT(*) AS fall_2026_entries
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026';
"""
```

WHAT THE QUERY DOES

This counts every row where the term is 'Fall 2026'. Capital letters and spaces at the start or end are ignored. The term column is the start term the applicant listed on their Grad Café entry.

Question 2

Among entries that provide a nationality classification, what percentage are international students?

RESULT

Percent international: **46.35%**

International entries: 13,885

Entries with a nationality classification: 29,955

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q2 = r"""
SELECT
    COUNT(*) FILTER (
        WHERE LOWER(TRIM(us_or_international)) = 'international'
    ) AS international_entries,
    COUNT(*) AS classified_entries,
    ROUND(100.0 * COUNT(*) FILTER (
        WHERE LOWER(TRIM(us_or_international)) = 'international'
    ) / NULLIF(COUNT(*), 0), 2) AS percent_international
FROM applicants
WHERE LOWER(TRIM(us_or_international)) IN ('international', 'american', 'other');
"""
```

WHAT THE QUERY DOES

The WHERE clause only keeps rows where the applicant chose International, American or Other, so entries with a missing or blank nationality are left out of the total. FILTER then counts how many of those rows are International. That count is divided by the total, multiplied by 100 and rounded to two decimals. American and Other entries count toward the total but not toward the international count. NULLIF stops the query from dividing by zero if no rows match.

Question 3

What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

RESULT

Average GPA: **3.77** (n = 18,288)
Average GRE Quantitative: 165.82 (n = 967)
Average GRE Verbal: 160.74 (n = 2,032)
Average GRE Analytical Writing: 4.35 (n = 1,829)

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q3 = r"""
SELECT
    ROUND((AVG(gpa) FILTER (WHERE gpa > 0 AND gpa <= 4.0))::numeric, 2) AS avg_gpa,
    COUNT(gpa) FILTER (WHERE gpa > 0 AND gpa <= 4.0) AS gpa_n,
    ROUND((AVG(gre) FILTER (WHERE gre BETWEEN 130 AND 170))::numeric, 2) AS avg_gre_quant,
    COUNT(gre) FILTER (WHERE gre BETWEEN 130 AND 170) AS gre_quant_n,
    ROUND((AVG(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170))::numeric, 2) AS avg_gre_verbal,
    COUNT(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170) AS gre_verbal_n,
    ROUND((AVG(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6))::numeric, 2) AS avg_gre_aw,
    COUNT(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6) AS gre_aw_n
FROM applicants;
"""
```

WHAT THE QUERY DOES

Each average has its own FILTER, so an applicant is counted for every score they reported, whether their other scores are blank, off the scale or valid. A score only counts when it is on the official scale: a GPA above 0 and up to 4.0, GRE Verbal and Quantitative from 130 to 170, and Analytical Writing from 0 to 6. A lot of applicants typed their total GRE score (260–340) into the Quantitative field, or 99.99 into Analytical Writing. Including those would

give averages that are not real scores. AVG skips empty (NULL) values, and ROUND(...::numeric, 2) rounds each average to two decimals.

Question 4

What is the average GPA of American applicants who applied for Fall 2026?

RESULT

Average GPA of American Fall 2026 applicants: **3.79**
Applicants with a GPA: 10,355

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q4 = r"""
SELECT
    ROUND(AVG(gpa)::numeric, 2) AS avg_gpa,
    COUNT(gpa)                  AS applicants_with_gpa
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND LOWER(TRIM(us_or_international)) = 'american'
    AND gpa > 0 AND gpa <= 4.0;
"""
```

WHAT THE QUERY DOES

This keeps only Fall 2026 entries from American applicants who reported a GPA on the 4.0 scale, then averages that GPA. COUNT(gpa) shows how many applicants the average is based on.

Question 5

What percentage of Fall 2025 entries are acceptances?

RESULT

Fall 2025 acceptance percentage: **48.22%**
Accepted Fall 2025 entries: 95
All Fall 2025 entries: 197

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q5 = r"""
SELECT
    COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')           AS accepted_entries,
    COUNT(*)                                                     AS fall_2025_entries,
    ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
          / NULLIF(COUNT(*), 0), 2)                               AS acceptance_percent
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2025';
"""
```

WHAT THE QUERY DOES

All Fall 2025 entries make up the total. FILTER counts the ones whose status starts with 'accept', and that count is divided by the total. The percentage is rounded to two decimals.

Question 6

What is the average GPA of accepted applicants who applied for Fall 2026?

RESULT

Average GPA of accepted Fall 2026 applicants: **3.76**

Applicants with a GPA: 7,322

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q6 = r"""
SELECT
    ROUND(AVG(gpa)::numeric, 2) AS avg_gpa,
    COUNT(gpa)                  AS applicants_with_gpa
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND TRIM(status) ILIKE 'accept%'
    AND gpa > 0 AND gpa <= 4.0;
"""
```

WHAT THE QUERY DOES

This keeps only Fall 2026 entries that were accepted and that reported a GPA on the 4.0 scale, then averages that GPA.

Question 7

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

RESULT

JHU Computer Science master's entries: **8**

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q7 = r"""
SELECT COUNT(*) AS jhu_cs_masters_entries
FROM applicants
WHERE program ~* '(johns?\s+hopkins|\mjhu\M)'
      AND program ~* '(computer\s+science|\meecs\M)'
      AND degree ~* '^s*master';
"""
```

WHAT THE QUERY DOES

This only uses the original fields downloaded from Grad Café. The program column holds text like 'Program, University', so one regular expression looks for Johns Hopkins (it also catches 'John Hopkins' and JHU as a whole word) and another looks for Computer Science (it also catches EECS). Both ignore case. The degree has to start with 'Master'.

Question 8

How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown University, MIT, Stanford University, or Carnegie Mellon University (using the original downloaded fields)?

RESULT

Accepted Fall 2026 CS PhD entries (original fields): **32**

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q8 = r"""
SELECT COUNT(*) AS accepted_cs_phd_entries
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
      AND TRIM(status) ILIKE 'accept%'
      AND degree ~* '^s*ph\.\?s*d'
      AND program ~* '(computer\s+science|\meecs\M)'
      AND (
        program ~* 'georgetown'
        OR program ~* 'massachusetts\s+institute\s+of\s+technology'
        OR program ~* '\mMIT\M'
        OR program ~* 'stanford'
        OR program ~* '(carnegie\s+mellon|\mcmu\M)');
"""
```

WHAT THE QUERY DOES

This applies five conditions at once. The term is Fall 2026, the status is an acceptance, the degree is a PhD (written PhD or Ph.D.), the program is Computer Science, and the school is one of the four universities, found in the original program text. Computer Science also matches 'Electrical Engineering and Computer Science', which is how MIT lists its CS PhD, and the EECS acronym. MIT is matched by its full name or by 'MIT' in capital letters, so words that only contain the letters 'mit' do not count.

Question 9

Repeating Question 8 with the LLM-generated program and university fields, how does the count compare with the original-field count?

RESULT

Original-field count: 32

LLM-field count: 32

Difference: 0

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q9 = r"""
SELECT
    COUNT(*) FILTER (
        WHERE program ~* '(computer\s+science|\meecs\M)'
        AND (
            program ~* 'georgetown'
            OR program ~* 'massachusetts\s+institute\s+of\s+technology'
            OR program ~* '\mMIT\M'
            OR program ~* 'stanford'
            OR program ~* '(carnegie\s+mellon|\mcmu\M)'
        ) AS original_field_count,
    COUNT(*) FILTER (
        WHERE llm_generated_program ~* '(computer\s+science|\meecs\M)'
        AND LOWER(TRIM(llm_generated_university)) IN ('georgetown university',
            'massachusetts institute of technology',
            'stanford university',
            'carnegie mellon university')
        ) AS llm_field_count
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND TRIM(status) ILIKE 'accept%'
    AND degree ~* '^\\s*ph\\.?\s*d';
"""
```

WHAT THE QUERY DOES

The WHERE clause keeps the same term, status and degree conditions as Question 8. Then FILTER computes both counts in one pass. The first count uses the original program text, exactly like Question 8. The second uses the program and university names produced by the LLM, and the whole university name, ignoring capital letters and spaces at the start or end, has to equal one of the four standard names. The difference is the LLM count minus the original count.

Why the two counts match: Grad Café's submission form now has applicants choose their university from a list, so these four schools already come in with one spelling each. The LLM step only removes the '(MIT)' at the end of MIT's name. The Computer Science program names are also short and consistent, so both methods pick out the same entries. The counts would differ for entries typed in by hand. A misspelled or unusual school or program name that the patterns above miss (for example 'Stanferd', 'Carnegie-Mellon', or 'CS' without the words 'Computer Science') would only be caught by the LLM fields if the LLM mapped it to the right name. On the other hand, if the LLM makes up a name or merges two schools by mistake, it can add or drop an entry that the original text gets right.

Question 10

Original question: For Fall 2026, how do the acceptance rate and the average GPA of accepted applicants compare across degree types that have at least 100 entries?

RESULT

Degree	Entries	Acceptances	Acceptance %	Avg GPA (accepted)
PhD	21,397	5,570	26.03%	3.81
Masters	7,515	5,128	68.24%	3.72
MFA	602	191	31.73%	3.73
PsyD	309	108	34.95%	3.84
Other	208	122	58.65%	3.71

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q10 = r"""
SELECT
    degree,
    COUNT(*)
    COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
    ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
    / NULLIF(COUNT(*), 0), 2)
    ROUND((AVG(gpa) FILTER (WHERE TRIM(status) ILIKE 'accept%'
    AND gpa > 0 AND gpa <= 4.0))::numeric, 2)
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
AND degree IS NOT NULL
GROUP BY degree
HAVING COUNT(*) >= 100
ORDER BY entries DESC, degree;
"""
```

WHAT THE QUERY DOES

This groups the Fall 2026 entries by degree type and keeps only the degree types with at least 100 entries (that is what HAVING does). For each group it shows the number of entries, how many were accepted, the acceptance percentage, and the average GPA of the accepted applicants who reported a GPA on the 4.0 scale.

Question 11

Original question: Which ten universities (LLM-standardized names) have the most Fall 2026 entries, and what percentage of each university's entries report an acceptance?

RESULT

University	Entries	Acceptances	Acceptance %
University of Michigan, Ann Arbor	803	232	28.89%
Stanford University	740	140	18.92%
University of California, Berkeley	690	176	25.51%
Yale University	592	85	14.36%
Columbia University	578	285	49.31%

Princeton University	571	121	21.19%
Massachusetts Institute of Technology	561	102	18.18%
University of Washington	553	156	28.21%
University of Wisconsin–Madison	543	103	18.97%
Harvard University	535	109	20.37%

SQL QUERY (AS RUN BY QUERY_DATA.PY)

```
SQL_Q11 = r"""
SELECT
    llm_generated_university           AS university,
    COUNT(*)                          AS entries,
    COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%') AS acceptances,
    ROUND(100.0 * COUNT(*) FILTER (WHERE TRIM(status) ILIKE 'accept%')
          / NULLIF(COUNT(*), 0), 2)   AS acceptance_percent
FROM applicants
WHERE LOWER(TRIM(term)) = 'fall 2026'
    AND llm_generated_university IS NOT NULL
    AND llm_generated_university <> 'Unknown'
GROUP BY llm_generated_university
ORDER BY entries DESC, university
LIMIT 10;
"""
```

WHAT THE QUERY DOES

This groups the Fall 2026 entries by the university name the LLM produced, which combines most of the different spellings of the same school. Rows where the LLM could not tell the school ('Unknown') are skipped. The groups are sorted by number of entries, the top ten are kept, and each one shows the percentage of its entries that report an acceptance. A few small groups stay separate (for example 'Yale' or 'Stanford' written by itself, each with fewer than ten rows), but they are too small to change which schools make the top ten.